

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : ANDHIKA NUR PRATAMA

NIM : 109082500056

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Sorting adalah proses mengurutkan sekumpulan data berdasarkan urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*), dengan tujuan mempermudah pengolahan, pencarian, dan analisis data. Pada Modul 14 dipelajari dua algoritma pengurutan, yaitu **Selection Sort** dan **Insertion Sort**. Selection Sort bekerja dengan cara mencari nilai terkecil atau terbesar dari bagian data yang belum terurut, kemudian menukarnya dengan elemen yang berada pada posisi yang seharusnya hingga seluruh data tersusun dengan benar. Sementara itu, Insertion Sort bekerja dengan cara mengambil satu elemen dari bagian yang belum terurut, lalu menyisipkannya ke posisi yang tepat pada bagian data yang sudah terurut dengan menggeser elemen lain jika diperlukan. Kedua algoritma ini dapat diterapkan pada berbagai tipe data, baik data sederhana seperti bilangan bulat maupun data bertipe *struct* seperti data mahasiswa dan buku, dengan pengurutan berdasarkan atribut tertentu, misalnya IPK atau rating. Selain digunakan untuk menyusun data agar lebih rapi dan terstruktur, proses sorting juga berperan penting dalam meningkatkan efisiensi pencarian data, terutama ketika dikombinasikan dengan metode pencarian seperti **Binary Search**, yang hanya dapat bekerja secara optimal pada data yang telah terurut. Oleh karena itu, sorting merupakan salah satu konsep dasar yang sangat penting dalam pemrograman karena membantu meningkatkan efektivitas pengelolaan dan pencarian data dalam suatu aplikasi.

B. Guided

1. [selSortArray]

```
package main

import "fmt"

func selectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr); i++ {
        idx_min = i
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}
```

```

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan angka index ke-%d: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    selectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output :

```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d
\Pertemuan12_Modul13\Guided\Guided-1\selSortArray
Masukkan angka index ke-0: 3
Masukkan angka index ke-1: 7
Masukkan angka index ke-2: 4
Masukkan angka index ke-3: 1
Masukkan angka index ke-4: 8
Masukkan angka index ke-5: 9
Masukkan angka index ke-6: 4
Masukkan angka index ke-7: 2

=== SEBELUM SORTING ===
3 7 4 1 8 9 4 2
=== SETELAH SORTING ===
1 2 3 4 4 7 8 9
PS D:\109082500056_Andhika-Nur-Pratama>

```

2. [selSortStruct]

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func selectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int

```

```

        for i = 0; i < len(sMHS)-1; i++ {
            idx_min = i
            for j = i + 1; j < len(sMHS); j++ {
                if sMHS[j].IPK < sMHS[idx_min].IPK {
                    idx_min = j
                }
            }
            if idx_min != i {
                sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
            }
        }
    }

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan data mahasiswa ke-%d: ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    selectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

PS D:\10908250056_Andhika-Nur-Pratama> g
atama\Pertemuan12_Modul13\Guided\Guided-2\
--- Masukkan data mahasiswa ke-0: ---
Nama : Umi
NIM : 123
IPK : 4.0
--- Masukkan data mahasiswa ke-1: ---
Nama : Mayuri
NIM : 456
IPK : 3.8
--- Masukkan data mahasiswa ke-2: ---
Nama : Nishi
NIM : 345
IPK : 3.9
--- Masukkan data mahasiswa ke-3: ---
Nama : Kaori
NIM : 678
IPK : 3.7
--- Masukkan data mahasiswa ke-4: ---
Nama : Sayaka
NIM : 567
IPK : 4.0

```

```

=== SEBELUM SORTING ===
--- Data mahasiswa ke-0 ---
Nama : Umi
NIM : 123
IPK : 4.00
--- Data mahasiswa ke-1 ---
Nama : Mayuri
NIM : 456
IPK : 3.80
--- Data mahasiswa ke-2 ---
Nama : Nishi
NIM : 345
IPK : 3.90
--- Data mahasiswa ke-3 ---
Nama : Kaori
NIM : 678
IPK : 3.70
--- Data mahasiswa ke-4 ---
Nama : Sayaka
NIM : 567
IPK : 4.00
=====

```

```

=== SETELAH SORTING ===
--- Data mahasiswa ke-0 ---
Nama : Kaori
NIM : 678
IPK : 3.70
--- Data mahasiswa ke-1 ---
Nama : Mayuri
NIM : 456
IPK : 3.80
--- Data mahasiswa ke-2 ---
Nama : Nishi
NIM : 345
IPK : 3.90
--- Data mahasiswa ke-3 ---
Nama : Umi
NIM : 123
IPK : 4.00
--- Data mahasiswa ke-4 ---
Nama : Sayaka
NIM : 567
IPK : 4.00
=====

```

3. [insertionDescending]

```

package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data : ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data : ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

```

```

    }

    fmt.Println("\nData sebelum sorting : ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)
    fmt.Println("\n\nData setelah sorting (Descending) : ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output :

```

PS D:\109082500056_Andhika-Nur-Pratama> go run "c:\
\Pertemuan12_Modul13\Guided\Guided-3\insertionDes
Masukkan jumlah data : 5
Masukkan data :
7 3 9 1 5

Data sebelum sorting :
7 3 9 1 5

Data setelah sorting (Descending) :
9 7 5 3 1
PS D:\109082500056_Andhika-Nur-Pratama>

```

4. [insertionStruct]

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

type arrMHS [100]mahasiswa

func insertionSortStruct(T *arrMHS, n int) {
    var temp mahasiswa
    var i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
    }
}

```

```

        T[j+1] = temp
    }
}

func main() {
    var data arrMHS
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa : ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData Mahasiswa ke-", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM : ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&data[i].IPK)
    }

    fmt.Println("\nData sebelum sorting : ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].IPK)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending
berdasarkan nama) : ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].IPK)
    }
}

```

Output :

```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d:\109082500056_Andhika-Nur-Pratama\
\Pertemuan12_Modul13\Guided\Guided-4\insertionStruct.go"
Masukkan jumlah mahasiswa : 3

Data Mahasiswa ke- 1
Nama : Ciaki
NIM : 111
IPK : 3.5

Data Mahasiswa ke- 2
Nama : Ayaka
NIM : 222
IPK : 3.9

Data Mahasiswa ke- 3
Nama : Yui
NIM : 333
IPK : 4

Data sebelum sorting :
Ciaki 111 3.5
Ayaka 222 3.9
Yui 333 4

Data setelah sorting (Descending berdasarkan nama) :
Yui 333 4
Ciaki 111 3.5
Ayaka 222 3.9
PS D:\109082500056_Andhika-Nur-Pratama>

```

C. Unguided

1. Soal Unguided 1

[rumahKerabat]

Jawaban :

```

package main

import "fmt"

const NMAX = 1000

type arrInt [NMAX]int

func selectionSort(A *arrInt, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

```



```

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func main() {
    var daerah, m int
    var rumah arrInt

    fmt.Scan(&daerah)

    for i := 0; i < daerah; i++ {

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

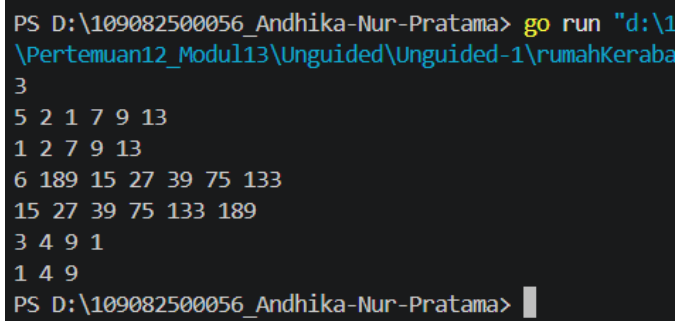
        selectionSort(&rumah, m)

        for j := 0; j < m; j++ {
            fmt.Print(rumah[j])

            if j < m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}

```

Output :



```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d:\1
\Pertemuan12_Modul13\Unguided\Unguided-1\rumahKerabat
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\109082500056_Andhika-Nur-Pratama>

```

Penjelasan :

[Program membaca nomor rumah kerabat pada setiap daerah lalu menyimpannya ke dalam array. Data diurutkan secara menaik menggunakan algoritma Selection Sort dengan mencari nilai terkecil pada bagian array yang

belum terurut. Setelah proses pengurutan selesai, nomor rumah dicetak secara ascending untuk setiap daerah.]

2. Soal Unguided 2

[kerabatDekat]

Jawaban :

```
package main

import "fmt"

const NMAX = 1000

type arrInt [NMAX]int

func selectionSort(A *arrInt, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func main() {
    var daerah, m, x int
    var ganjil, genap arrInt
    var ng, ne int

    fmt.Scan(&daerah)

    for d := 0; d < daerah; d++ {
        ng = 0
        ne = 0

        fmt.Scan(&m)

        for i := 0; i < m; i++ {
```

```

        fmt.Scan(&x)

        if x%2 == 1 {
            ganjil[ng] = x
            ng++
        } else {
            genap[ne] = x
            ne++
        }
    }

    selectionSort(&ganjil, ng)
    selectionSort(&genap, ne)

    for i := 0; i < ng; i++ {
        fmt.Print(ganjil[i], " ")
    }

    for i := ne - 1; i >= 0; i-- {
        fmt.Print(genap[i], " ")
    }

    fmt.Println()
}
}

```

Output :

```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d:
\Pertemuan12_Modul13\Unguided\Unguided-2\kerabatDe
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\109082500056_Andhika-Nur-Pratama> 

```

Penjelasan :

[Program memisahkan nomor rumah ganjil dan genap ke dalam dua array yang berbeda. Kedua array diurutkan menggunakan Selection Sort, kemudian nomor ganjil dicetak secara ascending dan nomor genap dicetak secara descending. Cara ini membuat Hercules dapat mengunjungi kerabat dengan jumlah penyeberangan jalan yang lebih sedikit.]

3. Soal Unguided 3

[DataBerjarakSama]

Jawaban :

```
package main

import "fmt"

const NMAX = 1000

type arrInt [NMAX]int

func insertionSort(A *arrInt, n int) {
    var i, j, temp int

    for i = 1; i < n; i++ {
        temp = A[i]
        j = i - 1

        for j >= 0 && A[j] > temp {
            A[j+1] = A[j]
            j--
        }

        A[j+1] = temp
    }
}

func main() {
    var A arrInt
    var x, n int

    n = 0

    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }

        A[n] = x
        n++
    }

    insertionSort(&A, n)

    for i := 0; i < n; i++ {
        fmt.Print(A[i], " ")
    }
    fmt.Println()
}
```

```

    jarakTetap := true
    selisih := A[1] - A[0]

    for i := 2; i < n; i++ {
        if A[i]-A[i-1] != selisih {
            jarakTetap = false
        }
    }

    if jarakTetap {
        fmt.Println("Data berjarak", selisih)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :

```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d:\
\Pertemuan12_Modul13\Unguided\Unguided-3\DataBerjar
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\109082500056_Andhika-Nur-Pratama> go run "d:\
\Pertemuan12_Modul13\Unguided\Unguided-3\DataBerjar
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\109082500056_Andhika-Nur-Pratama> 

```

Penjelasan :

[Program membaca bilangan bulat non-negatif hingga ditemukan bilangan negatif sebagai penanda akhir input. Seluruh data kemudian diurutkan menggunakan algoritma Insertion Sort dan diperiksa selisih antar elemen yang berurutan. Jika semua selisih sama maka program menampilkan "Data berjarak x", sedangkan jika berbeda akan menampilkan "Data berjarak tidak tetap".]

4. Soal Unguided 3

[DataBukuPerpustakaan]

Jawaban :

```

package main

import "fmt"

const NMAX = 7919

```

```

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [NMAX]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    var i int

    fmt.Scan(n)

    for i = 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var idxMax, i int

    idxMax = 0

    for i = 1; i < n; i++ {
        if pustaka[i].rating > pustaka[idxMax].rating {
            idxMax = i
        }
    }

    fmt.Println("=== Buku Terfavorit ===")
    fmt.Println("Judul      :", pustaka[idxMax].judul)
    fmt.Println("Penulis   :", pustaka[idxMax].penulis)
    fmt.Println("Penerbit  :", pustaka[idxMax].penerbit)
    fmt.Println("Tahun     :", pustaka[idxMax].tahun)
    fmt.Println()
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku
    var i, j int

    for i = 1; i < n; i++ {

        temp = pustaka[i]

```

```

        j = i - 1

        for j >= 0 && temp.rating > pustaka[j].rating {
            pustaka[j+1] = pustaka[j]
            j--
        }

        pustaka[j+1] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var batas int

    batas = 5
    if n < 5 {
        batas = n
    }

    fmt.Println("=== 5 Buku Rating Tertinggi ===")

    for i := 0; i < batas; i++ {
        fmt.Printf("%d. %s (Rating %d)\n",
            i+1,
            pustaka[i].judul,
            pustaka[i].rating)
    }

    fmt.Println()
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    var kiri, kanan, tengah int
    var ketemu bool

    kiri = 0
    kanan = n - 1
    ketemu = false

    for kiri <= kanan && !ketemu {

        tengah = (kiri + kanan) / 2

        if pustaka[tengah].rating == r {

            fmt.Println("=== Buku Ditemukan ===")
            fmt.Println("ID          :", pustaka[tengah].id)
            fmt.Println("Judul          :",
pustaka[tengah].judul)
            fmt.Println("Penulis       :",
pustaka[tengah].penulis)

```

```

        fmt.Println("Penerbit   :",
pustaka[tengah].penerbit)
        fmt.Println("Eksemplar  :",
pustaka[tengah].eksemplar)
        fmt.Println("Tahun      :",
pustaka[tengah].tahun)
        fmt.Println("Rating     :",
pustaka[tengah].rating)

        ketemu = true

        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if !ketemu {
        fmt.Println("Tidak ada buku dengan rating seperti
itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var n, ratingCari int

    DaftarkanBuku(&pustaka, &n)

    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)

    Cetak5Terbaru(pustaka, n)

    fmt.Scan(&ratingCari)

    CariBuku(pustaka, n, ratingCari)
}

```

Output :


```

PS D:\109082500056_Andhika-Nur-Pratama> go run "d:\109082500056_Andhika-Nur-Pratama\
\Pertemuan12_Modul13\Unguided\Unguided-4\DataBukuPer5
B01 LaskarPelangi AndreaHirata Bentang 10 2005 95
B02 Bumi Tereliye Gramedia 7 2014 90
B03 AtomicHabits JamesClear Penguin 12 2018 98
B04 FilosofiTeras HenryManampiring Kompas 5 2018 88
B05 DeepWork CalNewport Piatkus 8 2016 92
=== Buku Terfavorit ===
Judul   : AtomicHabits
Penulis  : JamesClear
Penerbit : Penguin
Tahun   : 2018

=== 5 Buku Rating Tertinggi ===
1. AtomicHabits (Rating 98)
2. LaskarPelangi (Rating 95)
3. DeepWork (Rating 92)
4. Bumi (Rating 90)
5. FilosofiTeras (Rating 88)

92
=== Buku Ditemukan ===
ID      : B05
Judul   : DeepWork
Penulis  : CalNewport
Penerbit : Piatkus
Eksemplar : 8
Tahun   : 2016
Rating  : 92
PS D:\109082500056_Andhika-Nur-Pratama>

```

Penjelasan :

[Program menyimpan data buku ke dalam array bertipe struct yang berisi informasi buku seperti judul, penulis, penerbit, tahun, dan rating. Data buku diurutkan berdasarkan rating tertinggi menggunakan algoritma Insertion Sort, kemudian ditampilkan buku terfavorit dan lima buku dengan rating tertinggi. Program juga menggunakan Binary Search untuk mencari buku berdasarkan rating yang dimasukkan pengguna.]

D. Kesimpulan

Berdasarkan seluruh latihan pada Modul 14, dapat disimpulkan bahwa algoritma **Selection Sort** dan **Insertion Sort** merupakan metode pengurutan data yang penting dalam pemrograman. Pada latihan pertama, Selection Sort digunakan untuk mengurutkan nomor rumah kerabat secara menaik serta mengelompokkan nomor rumah ganjil dan genap sesuai kebutuhan permasalahan. Sementara itu, pada latihan kedua, Insertion Sort digunakan untuk mengurutkan data bilangan bulat dan data buku berdasarkan atribut tertentu, seperti rating. Selain melakukan pengurutan, program juga menerapkan analisis data, seperti pemeriksaan jarak antar bilangan dan pencarian data buku menggunakan metode **Binary Search**. Melalui latihan-latihan tersebut, dapat dipahami bahwa proses sorting sangat membantu dalam menyusun data agar lebih terstruktur, mempermudah proses pencarian, serta

meningkatkan efisiensi pengolahan data pada berbagai jenis aplikasi, baik yang menggunakan data sederhana maupun data bertipe *struct*.